

# CSSCM 2026 COHORT

## NumPy Quick Reference

### Arrays and Multidimensional Arrays - Student Cheat Sheet

|          |                                                |
|----------|------------------------------------------------|
| Director | Elliot S. Menkah (PhD)                         |
| Tutors   | Emmanuel Oppong (MPhil); Obed Adu-Gyamfi (BSc) |

**Purpose:** Syntax reference only. This sheet does not contain answers to the chemistry practical.

## 1. Import NumPy

```
import numpy as np
```

The alias np is the standard short name used throughout the workshop.

## 2. Create Arrays

```
# 1D array
x = np.array([2.0, 4.0, 6.0, 8.0])

# 2D array
M = np.array([[1.0, 2.0, 3.0],
              [4.0, 5.0, 6.0]])

# Generated arrays
a = np.arange(0, 10, 2)
b = np.linspace(0, 1, 6)
```

A 1D array behaves like a numerical vector. A 2D array has rows and columns.

## 3. Inspect an Array

```
print(x.ndim)    # number of dimensions
print(x.shape)   # size along each dimension
print(x.size)    # total number of elements
print(x.dtype)   # stored data type
```

For a matrix, shape is typically (rows, columns).

## 4. Indexing

```
x[0]      # first item
x[-1]     # last item

M[0, 0]   # row 0, column 0
M[1, 2]   # row 1, column 2
```

Python starts counting from index 0. Negative indices count from the end.

## 5. Slicing

```
x[1:4]     # positions 1, 2, 3
M[:, 0]    # every row, first column
M[0, :]    # first row, every column
M[0:2, 1:3] # selected rows and columns
```

The colon : means a range. A missing bound means 'from the beginning' or 'to the end'.

## 6. Vectorised Arithmetic

```
x + 2
x * 3
x ** 2
x / 10

y = np.array([1.0, 2.0, 4.0, 8.0])
```

```
ratio = x / y
```

NumPy can apply one operation to every element without writing a loop.

## 7. Useful Mathematical Functions

```
np.log(x)      # natural logarithm
np.exp(x)      # exponential
np.sqrt(x)     # square root
np.mean(x)
np.min(x)
np.max(x)
np.std(x)
```

These functions operate directly on NumPy arrays.

## 8. Understanding axis

```
# For a 2D matrix M:
np.mean(M, axis=0) # one result per column
np.mean(M, axis=1) # one result per row
np.mean(M)         # one overall mean
```

**Memory aid:** `axis=0` collapses the rows and leaves one value per column; `axis=1` collapses the columns and leaves one value per row.

## 9. Boolean Masks

```
mask = x > 4
print(mask)
print(x[mask])

count = np.sum(mask)
```

A comparison creates an array of True/False values. Use the mask to select matching data.

## 10. Loops and Conditions

```
for value in x:
    if value < 3:
        print("Low")
    elif value < 7:
        print("Medium")
    else:
        print("High")
```

Loops are useful when each value needs a separate decision or printed label.

## 11. Matrix Multiplication

```
A = np.array([[1, 2],
              [3, 4]])
v = np.array([5, 6])

result1 = A @ v
result2 = np.dot(A, v)
```

The `@` operator and `np.dot()` can perform matrix-vector multiplication.

## 12. Solve a Linear System

```
A = np.array([[2.0, 1.0],
              [1.0, 3.0]])
b = np.array([5.0, 7.0])

x = np.linalg.solve(A, b)
print(x)

# Check the solution
print(A @ x)
print(b)
```

For a system  $\mathbf{Ax} = \mathbf{b}$ , `np.linalg.solve(A, b)` returns the unknown vector  $\mathbf{x}$ . Verify by calculating  $\mathbf{A} @ \mathbf{x}$ .

## Common Reminders

Use matching brackets. Keep indentation consistent. Array rows must normally have compatible lengths. Use decimal points when you want floating-point data. Read error messages from the bottom upward to find the immediate cause.